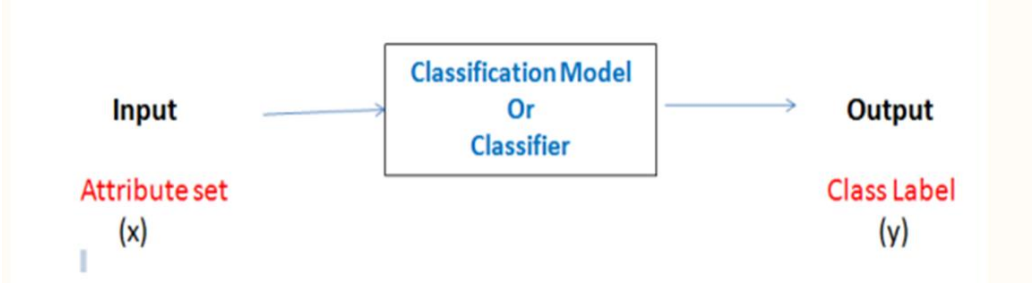
UNIT-3

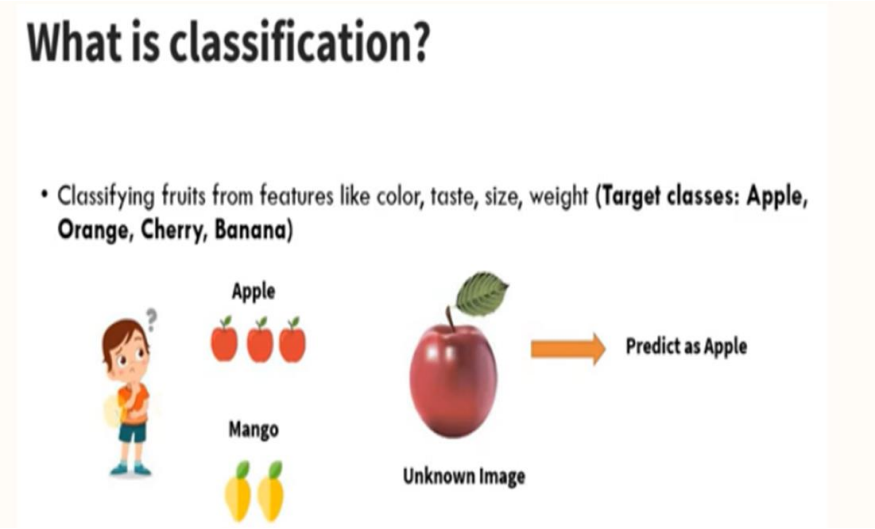
Classification: Basic Concepts, General Approach to solving a classification problem, Decision Tree Induction: Attribute Selection Measures, Tree Pruning, Scalability and Decision Tree Induction, Visual Mining for Decision Tree Induction, Bayesian Classification Methods: Bayes Theorem, Naïve Bayes Classification, Rule-Based Classification, Model Evaluation and Selection.

3.1 BASIC CONCEPTS

3.1.1 WHAT IS CLASSIFICATION?

- Classification is one form of data analysis, where a model or classifier is constructed to predict categorical labels, such as “safe” or “risky” for the loan application data





3.1.2 GENERAL APPROACH TO SOLVING A CLASSIFICATION PROBLEM

General Approach to Classification

- Data classification is a two-step process, consisting of a learning step (where a classification model is constructed) and a classification step (where the model is used to predict class labels for given data).

First step:(Learning) /Training the classifier/Model construction

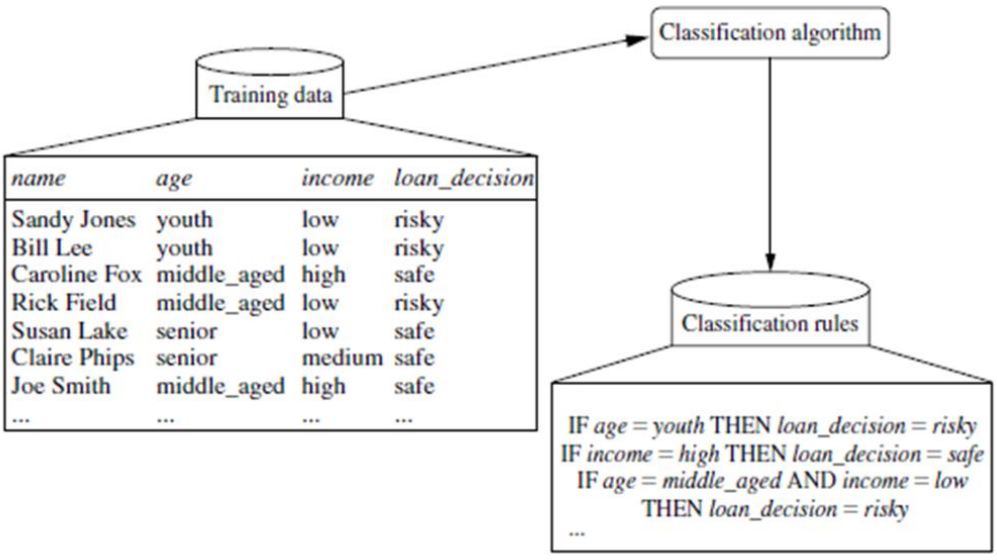
- A classifier is built describing a predetermined set of data classes. This is the learning step (or training phase), where a classification algorithm builds the classifier by analyzing or “learning from” a training set.

Second step :(Classification) /Testing the classifier/model usage

- The model is used for classification. Use the test set to measure the accuracy of the classifier

- **Ex :** A bank loans officer needs analysis of her data in order to learn which loan applicants are “safe”and which are “risky” for the bank.

The data classification process:



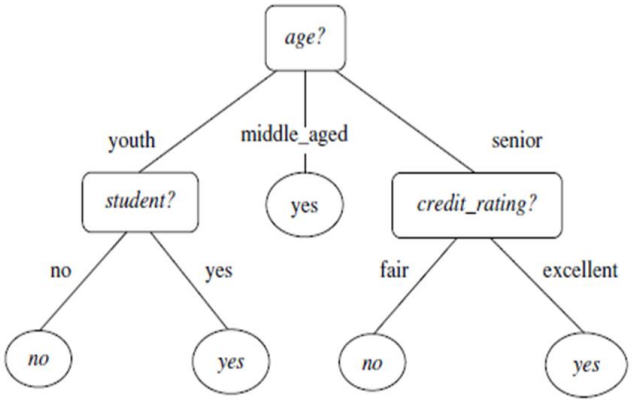
(a)

- The individual tuples making up the training set are referred to as training tuples and are randomly sampled from the database under analysis.
- In the context of classification, data tuples can be referred to as samples, examples, instances, data points, or objects.
- The class label of each training tuple is provided, this step is also known as supervised learning.
- It contrasts with unsupervised learning (or clustering), in which the class label of each training tuple is not known, and the number or set of classes to be learned may not be known in advance.

3.2 DECISION TREE INDUCTION

- **Decision tree induction** is the learning of decision trees from class-labeled training tuples.
- A **decision tree** is a flowchart-like tree structure, where each **internal node** (non leaf node) denotes a test on an attribute, each branch represents an outcome of the test, and each **leaf node** (or terminal node) holds a class label.
- The topmost node in a tree is the **root** node.

A decision tree for the concept buys computer



- It represents the concept buys computer, that is, it predicts whether a customer at AllElectronics is likely to purchase a computer.

- Internal nodes are denoted by rectangles, and leaf nodes are denoted by ovals.
- **ID3 (Iterative Dichotomiser)** & Classification and Regression **Trees (CART)** adopt a greedy approach in which decision trees are constructed in a **top-down recursive divide-and-conquer manner.**
- Most algorithms for decision tree induction is a top-down approach, which starts with a **training set of tuples** and their **associated class labels.**
- The training set is recursively partitioned into smaller subsets as the tree is being built.

- **3.2.1 BASIC DECISION TREE INDUCTION ALGORITHM**

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition, *D*.

Input:

- Data partition, *D*, which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split-point* or *splitting subset.*

Output: A decision tree.

Method:

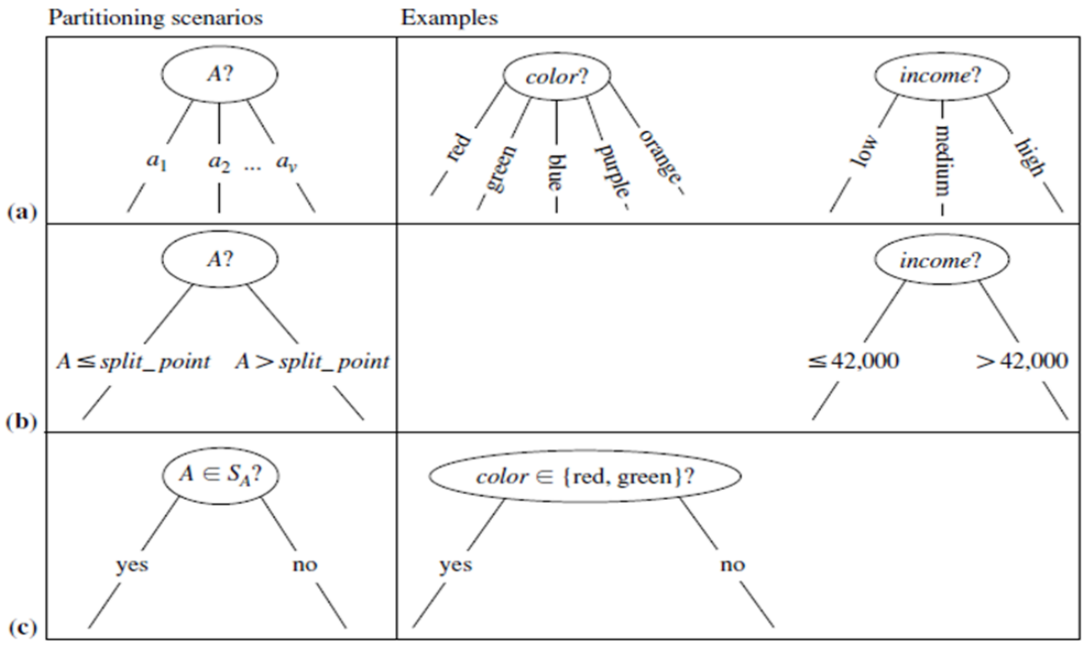
```
(1)  create a node N;
(2)  if tuples in D are all of the same class, C, then
(3)      return N as a leaf node labeled with the class C;
(4)  if attribute_list is empty then
(5)      return N as a leaf node labeled with the majority class in D; // majority voting
(6)  apply Attribute_selection_method(D, attribute_list) to find the "best" splitting_criterion;
(7)  label node N with splitting_criterion;
(8)  if splitting_attribute is discrete-valued and
         multiway splits allowed then // not restricted to binary trees
(9)      attribute_list ← attribute_list − splitting_attribute; // remove splitting_attribute
(10) for each outcome j of splitting_criterion
     // partition the tuples and grow subtrees for each partition
(11)     let Dj be the set of data tuples in D satisfying outcome j; // a partition
(12)     if Dj is empty then
(13)         attach a leaf labeled with the majority class in D to node N;
(14)     else attach the node returned by Generate_decision_tree(Dj, attribute_list) to node N;
     endfor
(15) return N;
```

The strategy is as follows.

- The algorithm is called with three parameters: **D, attribute list, and Attribute selection** method.
- We refer to D as a **data partition.**
- The parameter **attribute list** is a list of attributes describing the tuples.
- **Attribute selection** method employs an attribute selection measure such as information gain or the **Gini index.**
- The tree starts as a single node, N, representing the training tuples in D (step 1).
- If the tuples in D are all of the same class, then node N becomes a leaf and is labeled with that class (steps 2 and 3).

- Otherwise, the algorithm calls Attribute selection method to determine the **splitting criterion.** The splitting criterion tells us which attribute to test at node N by determining the "best" way to separate or partition the tuples in D into individual classes (step 6).
- The node N is labeled with the splitting criterion, which serves as a test at the node (step 7).
- A branch is grown from node N for each of the outcomes of the splitting criterion. The tuples in D are partitioned accordingly (steps 10 to 11).
- There are three possible scenarios,Let A be the splitting attribute. A has v distinct values, a1, a2, : : : , av based on the training data.
- **A is discrete-valued:** In this case, the outcomes of the test at node N correspond directly to the known values of A.
- **A is continuous-valued:** In this case, the test at node N has two possible outcomes, corresponding to the conditions A split point and A > split point, respectively,
- **A is discrete-valued** and a binary tree must be produced

- Three possibilities for partitioning tuples based on the splitting criterion



3.2.2 ATTRIBUTE SELECTION MEASURES: INFORMATION GAIN, GAIN RATIO

Attribute Selection Measures

- An attribute selection measure is a heuristic for selecting the splitting criterion that "best" separates a given data partition, D, of class-labeled training tuples into individual classes.
- If we were to split D into smaller partitions according to the outcomes of the splitting criterion, ideally each partition would be pure.

This section describes three popular attribute selection measures:

1. information gain
2. gain ratio
3. gini index

1. INFORMATION GAIN

- Information gain is defined as the difference between the original information requirement (i.e., based on just the proportion of classes) and the new requirement (i.e., obtained after partitioning on A). That is,

$$Gain(A) = Info(D) - Info_A(D).$$

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i),$$

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j).$$

Gain Ratio for Attribute Selection (C4.5)

- Information gain measure is biased towards attributes with a large number of values
- C4.5 (a successor of ID3) uses gain ratio to overcome the problem (normalization to information gain)

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right)$$

GainRatio(A) = Gain(A) / SplitInfo(A)

SPLIT BASED ON VARIABLE INCOME

	BUYS_COMP=YES	BUYS_COMP=NO	TOTAL
HIGH	2	2	4
MEDIUM	4	2	6
LOW	3	1	4
TOTAL	9	5	14

- For the previous example, if A = income then

$$SplitInfo_A(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right) = 0.926$$

GainRatio(income) = 0.029/0.926 = 0.031

- The attribute with the maximum gain ratio is selected as the splitting attribute

Table 8.1 Class-Labeled Training Tuples from the *AllElectronics* Customer Database

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculating Entropy or Info(D) for class label

Info(9,5)= - 9/14 log2(9/14)-5/14log2(5/14)

= -0.642*(log(0.642)/log2)-0.3571*(log(0.3571)/log2)

= -0.642*(-0.207/0.3010)-0.3571*(-0.447/0.3010)

=0.940bits

INFORMATION CALCULATED ON AGE ATTRIBUTE

SPLIT BASED ON VARIBALE AGE

	BUYS COMP=YES	BUYS_COMP=NO	TOTAL
YOUTH	2	3	5
MIDDLE_AGE	4	0	4
SENIOR	3	2	5
TOTAL	9	5	14

Info AGE(D)=5/14[Info(2,3)]*4/14[Info(4,0)]+5/14[Info(3,2)]

5/14[-2/5*log2(2/5)-3/5*log2(3/5)]+4/14(-4/4log2(4/4)-0/4log2(0/4]+5/14[-3/5log2(3/5)-2/5log2(2/5)]

=0.693bits

Gain(age)= Info(D)- Info AGE(D)

=0.940-0.693

0.247bits

INFORMATION CALCULATED ON INCOME ATTIBUTE

SPLIT BASED ON VARIABLE INCOME

	BUYS COMP=YES	BUYS_COMP=NO	TOTAL
HIGH	2	2	4
MEDIUM	4	2	6
LOW	3	1	4
TOTAL	9	5	14

InfoINCOME(D)=4/14*[-2/4log2(2/4)-2/4log2(2/4)]+6/14* (-4/6log2(4/6)-2/6log2(2/6)]+4/14[-3/4log2(3/4)-1/4log2(1/4)]

=0.911bits

Gain(income)= Info(D)- Info income(D)

0.940-0.911

=0.029bits

INFORMATION CALCULATED ON STUDENT ATTRIBUTE

SPLIT BASED ON VARIBALE STUDENT

	BUYS COMP=YES	BUYS_COMP=NO	TOTAL
NO	3	4	7
YES	6	1	7
TOTAL	9	5	14

Info student(D)=7/14[-3/7log2(3/7)-4/7log2(4/7)]+7/14(-6/7log2(6/7)-1/7log2(1/7)]

=0.789bits

Gain(student)=Info(D)- Info student(D)

=0.940-0.789

0.151bits

INFORMATION CALCULATED ON CREDIT-RATING

SPLITE BASED ON VARIBLE CREDIT_RATING

	BUYS COMP=YES	BUYS_COMP=NO	TOTAL
EXCELLENT	3	3	6
FAIR	6	2	8
TOTAL	9	5	14

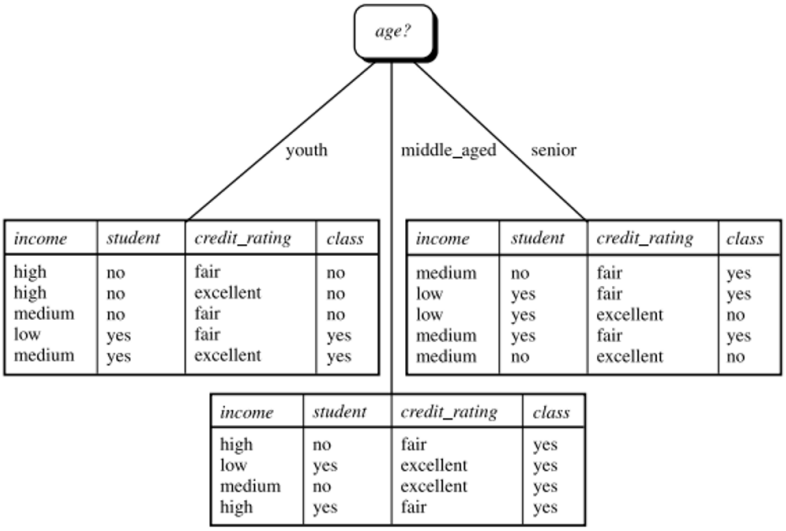
Info Credit_rating(D)=6/14[-3/6log2(3/6)-3/6log2(3/6)]+8/14[-6/8log2(6/8)-2/8log2(2/8)+

=0.892bits

Gain(credit_Rating)= Info(D)- Info Credit_rating(D)

=0.940-0.892

0.048bits



income	*student*	*credit_rating*	*class*
high	no	fair	no
high	no	excellent	no
medium	no	fair	no
low	yes	fair	yes
medium	yes	excellent	yes

income	*student*	*credit_rating*	*class*
medium	no	fair	yes
low	yes	fair	yes
low	yes	excellent	no
medium	yes	fair	yes
medium	no	excellent	no

income	*student*	*credit_rating*	*class*
high	no	fair	yes
low	yes	excellent	yes
medium	no	excellent	yes
high	yes	fair	yes

Steps are repeatedLeft-Side: Youth

Info(2,3)=-2/5log2(2/5)-3/5log2(3/5)=0.97

INFORMATION IS CALCULATED ON INCOME ATTRIBUTE

SPLITE BASED ON VARIBALE INCOME

	PLAY=YES	PLAY=NO	TOTAL
HIGH	0	2	2
LOW	1	0	1
MEDIUM	1	1	2
TOTLA	2	3	5

Infoincome(D)=2/5[Info(0,2)]+1/5[Info(1,0)+2/5[Info(1,1)]

=0.4

Gain(income)=Info(D)- Infoincome(D)

=0.97-0.4

0.570bits

INFORMATION IS CALCULATED ON STUDENT ATTRIBUTE

SPLITE BASED ON VARIBALE STUDENT

	PLAY=YES	PLAY=NO	TOTAL
NO	0	3	3
YES	2	0	2
TOTLA	2	3	5

Info student(D)=3/5[Info(0,3)]+2/5[Info(2,0)]=0

Gain(student)=Info(D)- Info student(D)=0.97-0

0.97bits

INFORMATION IS CALCULATED ON CERDIT_RATEING ATTRIBUTE

SPLITE BASED ON VARIBALE CREDIT_RATEIN G

	PLAY=YES	PLAY=NO	TOTAL
FAIR	1	2	3
EXCELLENT	1	1	2
TOTLA	2	3	5

Info Credit_Rating(D)=3/5[Info(1,2)]+2/5(Info(1,1)]=0.9508

Gain(Credit_Rating)=Info(D)- Info Credit_Rating(D)

=0.97-0.9508

0.0192bits

Right Side-Senior

Info(D)=Info(3,2)=-3/5log2(3/5)-2/5log2(2/5)=0.97bits

INFORMATION IS CALCULATED ON INCOME ATTRIBUTE

SPLITE BASED ON VARIBALE INCOME

	PLAY=YES	PLAY=NO	TOTAL
LOW	1	1	2
MEDIUM	2	1	3
TOTLA	3	2	5

Info income(D)=2/5[info(1,1)]+3/5(Info(2,1)]=0.9508

Gain(income)=Info(D)- Info income(D)=0.97-0.9508

0.0192bits

INFORMATION IS CALCULATED ON STUDENT ATTRIBUTE

SPLITE BASED ON VARIBALE STUDENT

	PLAY=YES	PLAY=NO	TOTAL
NO	1	1	2
YES	2	1	3
TOTLA	3	2	5

Info student(-D)=2/5[info(1,1)]+3/5(Info(2,1)]

=0.9508

Gain(student)=Info(D)- Info student(D)

=0.97-0.9508

0.0192bits

INFORMATION IS CALCULATED ON CREDIT_RATING ATTRIBUTE

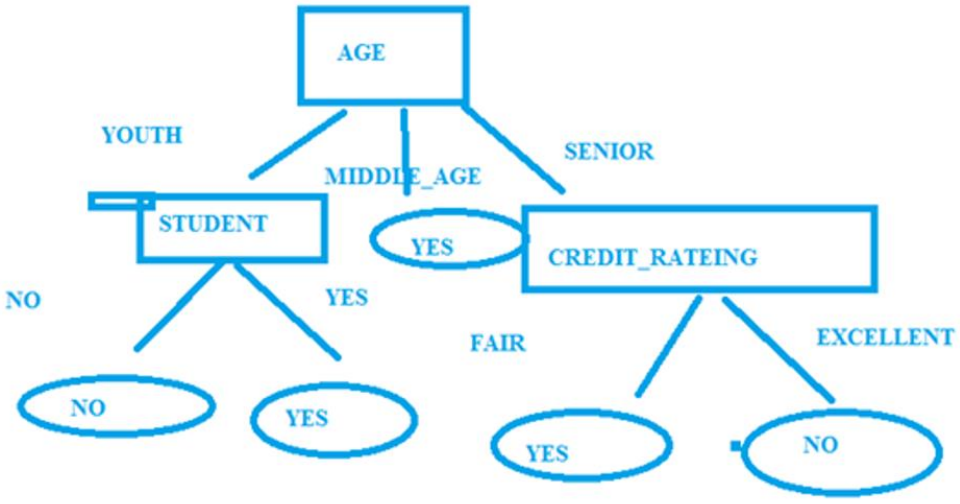
SPLITE BASED ON VARIBALE CREDIT_RATING

	PLAY=YES	PLAY=NO	TOTAL
FAIR	0	2	2
EXCELLENT	3	0	3
TOTLA	3	2	5

Info Credit_Rating(D)=2/5[Info(2,0)]+3/5(Info(3,0)] =0

Gain(Credit_Rating)=Info(D)- Info Credit_Rating(D)=0.97-0

0.97bits



ID3 WORKS ONLY FOR NOMINAL – DATA SET

3.2.3 TREE PRUNING

What is Tree Pruning?

- Tree pruning is the process of removing branches of a decision tree that are noisy, unreliable, or unnecessary to prevent overfitting.

Why Prune a Tree?

- Prevents overfitting.
- Reduces complexity.
- Improves accuracy on **unseen data.**
- Makes the model easier to interpret.

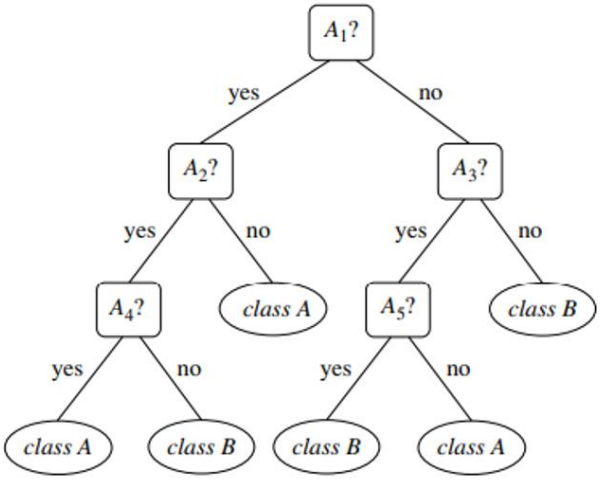


Figure : An unpruned decision tree

Two Main Types of Pruning

Type	When It Happens	How It Works
Prepruning	During tree building	Stop splitting early if the gain is too small
Postpruning	After tree is built	Cut subtrees that don't improve accuracy

1)Prepruning

- Stops tree growth early based on:
- Information Gain
- Gini Index
- Statistical Significance

Risk: Can stop too early and miss important patterns.

2)Postpruning

- Prunes the full-grown tree bottom-up.
- Replaces subtrees with a leaf (most common class).
- More reliable than prepruning.

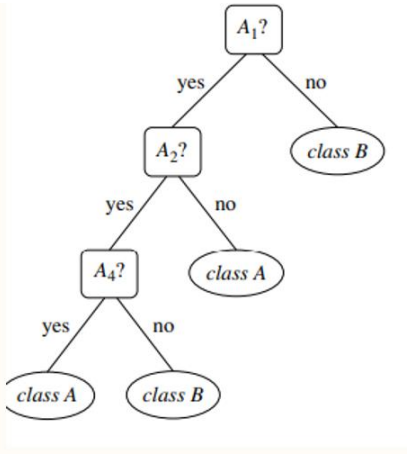


Figure: Pruning removes the subtree under A3 and replaces it with a single class leaf.

Postpruning Techniques

Method	Description
Cost Complexity (CART)	Prune if total error + tree size cost reduces. Needs a separate prune set.
Pessimistic Pruning (C4.5)	Uses training error + penalty (no prune set needed).
MDL (Minimum Description Length)	Chooses tree needing fewest bits to encode.

Issues in Large Trees

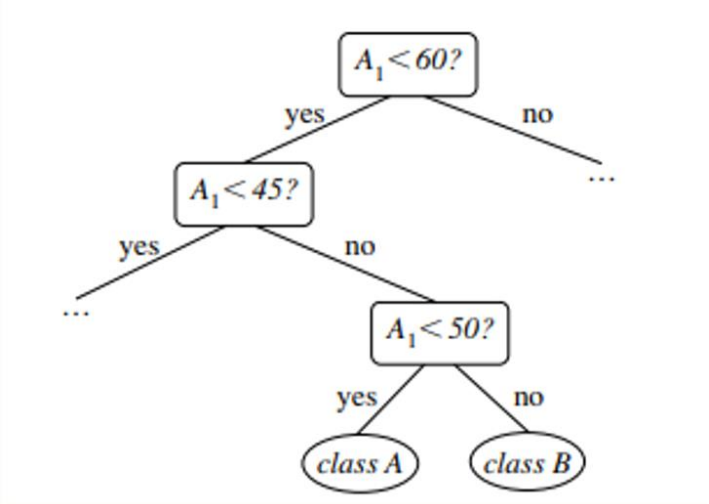
1)**Repetition**

2)**Replication**

These hurt clarity and accuracy.

1) Repetition

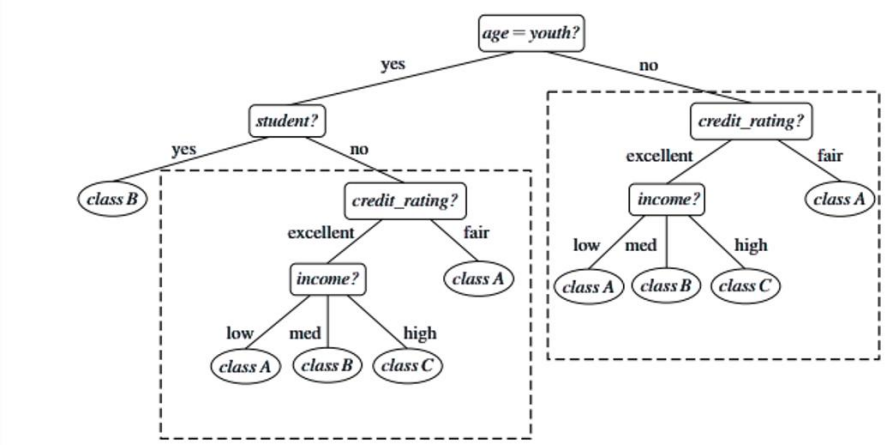
- The same attribute is tested multiple times along a single branch
- **Problem:** Redundant tests make the tree unnecessarily deep and hard to understand.



Example : subtree repetition, where an attribute is repeatedly tested along a given branch of the tree (e.g., age)

2) Replication

- The same subtree structure is repeated in multiple parts of the tree.
- **Problem:** This leads to a bloated tree with duplicate logic, making it inefficient and confusing.



Example: subtree replication, where duplicate subtrees exist within a tree (e.g., the subtree headed by the node "credit rating?")

Solutions to These Issues

1. Multivariate Splits

- Use a combination of attributes for splitting.
- **Example**: If (Age < 45 AND Income = High) → Class A
- Reduces depth and captures complex patterns in one node

2. Rule-Based Models

- Convert tree paths into if-then rules.
- **Example:**

Rule 1: If Age < 45 AND Student = Yes → Class A

Rule 2: If Age ≥ 45 AND Credit = Fair → Class B

- Easier to read, avoids redundancy, and focuses on logic.
- **Repetition** and **replication** make decision trees harder to interpret and can affect prediction accuracy.
- Using **multivariate splits** or transforming the tree into a **set of rules** makes the model more compact, clear, and efficient.

- **3.2.4 Scalability and Decision Tree Induction**

Problem: Traditional Algorithms Don't Scale

- Algorithms like ID3, C4.5, and CART assume the entire dataset fits in memory.
- In real-world applications, datasets are too large to fit in memory.
- This causes inefficiency due to constant disk-memory swapping.

Solution 1: RainForest Algorithm

What it does:

- It keeps only summary statistics in memory using AVC-sets instead of full data.

AVC-set (Attribute-Value-Classlabel)

- For each attribute value, it stores a count of how often each class label appears.
- Stored as a small data structure that fits in memory.

age	*buys_computer* yes	no
youth	2	3
middle_aged	4	0
senior	3	2

income	*buys_computer* yes	no
low	3	1
medium	4	2
high	2	2

student	*buys_computer* yes	no
yes	6	1
no	3	4

credit_ratting	*buys_computer* yes	no
fair	6	2
excellent	3	3

AVC-Set Based Data Summarization for Scalable Decision **Tree Induction**

Benefits of RainForest:

- Efficient and scalable even for large datasets
- Works with any decision tree algorithm (like C4.5, CART)
- Memory-adaptive

Solution 2: **BOAT (Bootstrapped Optimistic Algorithm for Tree Construction)**

What it does:

- Uses bootstrap sampling: picks small subsets from the large dataset that fit in memory.
- Builds a tree from each sample, then merges them into a final tree.

Benefits of BOAT:

- Very fast: Needs only 2 scans of the dataset.
- No special data structures needed.
- **Incremental updates:** Can update tree without full rebuild.

Feature	RainForest	BOAT
Strategy	Summarize data with AVC-sets	Use sampled subsets to build & merge trees
Memory Use	Low – stores summaries only	Low – samples fit in memory
Speed	Fast	Very Fast (2–3× faster than RainForest)
Incremental?	No	Yes – supports updates

3.2.5 Visual Mining for Decision Tree Induction

- Visual Mining lets users interactively build a decision tree by visualizing the data and using human insight to guide splits.

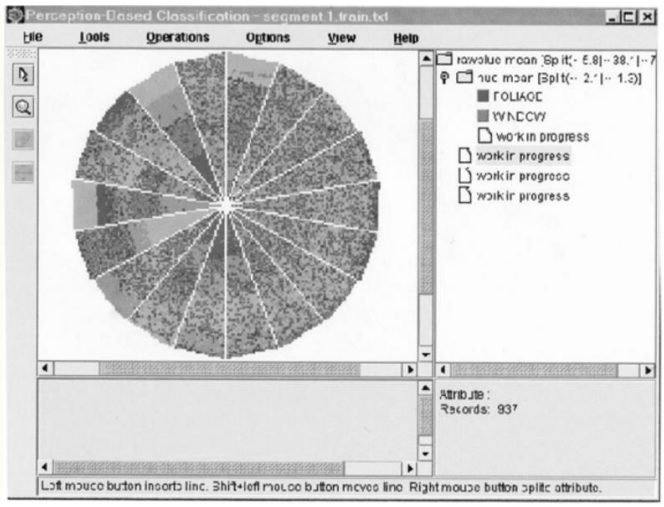
- **Perception-Based Classification (PBC)**

PBC is a visual, interactive method for decision tree construction where:

- Data is displayed as colored circle segments.
- Each segment represents an attribute.
- Each pixel shows the value and class of a data point

- **The PBC Interface Has Two Main Views:**

Window	Function
Data Interaction	Shows multidimensional data in circle segments
Knowledge Interaction	Shows the decision tree being constructed



Interactive Decision Tree Construction using PBC Interface

How It Works: Step-by-Step

- Visualize data → User sees circle segments based on attributes and class labels.
- Select attribute + split-points → User decides how to split data (can be multiple splits, unlike traditional binary-only).
- Tree node updates in the right window.
- Repeat the process until all leaves are assigned a class.

Key Features of PBC

Feature	Description
Interactive	User selects splits based on visual patterns
Multidimensional visualization	Helps understand complex data relationships
Multiple splits	One node can split into more than 2 branches
Smaller trees	Easier to understand, but still accurate
Domain knowledge use	Users can apply their own expertise while building the tree

Advantages Over Traditional Methods:

PBC vs Traditional (C4.5, CART)

- Smaller, simpler trees
- Similar accuracy
- Easier interpretation
- Supports user insight

PBC makes decision tree induction interactive and visual, allowing users to:

- Understand their data better,
- Incorporate expert knowledge,
- Build simpler and effective decision trees.

3.3 BAYESIAN CLASSIFICATION METHODS

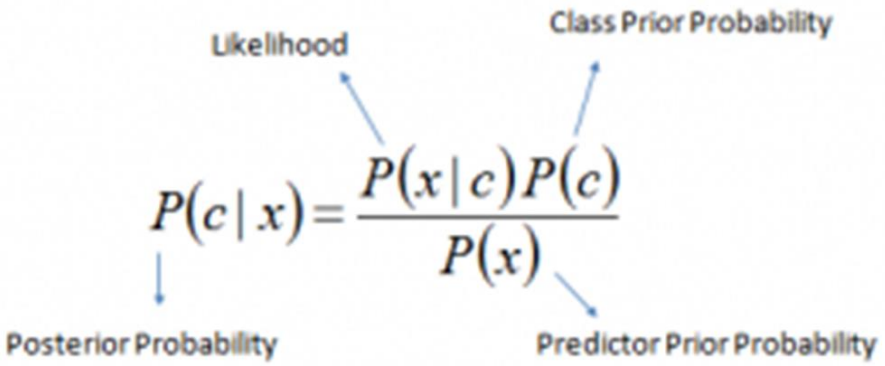
- Bayesian classifiers are statistical classifiers.
- They can predict class membership probabilities, such as the probability that a given tuple belongs to a particular class.
- Bayesian classification is based on Bayes' theorem
- Naïve Bayesian classifiers assume that the effect of an attribute value on a given class is **independent** of the values of the other attributes.
- This assumption is called class **conditional independence**

3.3.1 Bayes Theorem

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}.$$

- Let X be a data sample whose class label is unknown.
- Let H be some hypothesis, such as that the data sample X belongs to a specified class C.
- For classification problems, determine P (H/X), the probability that the hypothesis H holds given the observed data sample X
- P (H/X) is the posterior probability, of H conditioned on X
- P (H) is the prior probability of H.

- P(X/H) is the posterior probability/Conditional probability/Likelihood of X conditioned on H.



$$P(c \mid \mathbf{X}) = P(x_1 \mid c) \times P(x_2 \mid c) \times \cdots \times P(x_n \mid c) \times P(c)$$

3.3.2 Naïve Bayes Classification

- The naïve Bayesian classifier, or simple Bayesian classifier, works as follows:

1.Let D be a training set of tuples and their associated class labels.

- Each tuple is represented by an n-dimensional attribute vector, X = (x1, x2, : : : , xn), from n attributes, respectively, A1, A2, : : : , An.

2. Suppose that there are m classes, C1, C2, Cm.

- Given a tuple, X, the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X.
- naïve Bayesian classifier predicts that tuple X belongs to the class Ci if and only if P(Ci/X) > P(Cj/X) for $1 \leq j \leq m; j \neq i$
- Thus we maximize P(Ci/X). The class Ci for which P(Ci/X) is maximized is called the **maximum posteriori hypothesis.**

By Bayes' theorem

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}$$

3. As P(X) is constant for all classes, only P(X/Ci)P(Ci) need be maximized.

4. Given data sets with many attributes, it would be computationally expensive to compute P(X/Ci). In order to reduce computation in evaluating P(X/Ci), the naive assumption of **class conditional independence** i.e., that there are no dependence relationships among the attributes.

$$P(X|C_i) = \prod_{k=1}^{n} P(x_k|C_i)$$
$$= P(x_1|C_i) \times P(x_2|C_i) \times \cdots \times P(x_n|C_i)$$

- We can easily estimate the probabilities P(x1/Ci), P(x2/Ci), : : : , P(xn/Ci) from the training tuples.
- To compute P(X/Ci), we consider the following:

a)If Ak is categorical, then P(xk/Ci) is the number of tuples of class Ci in D having the value xk for Ak, divided by, the number of tuples of class Ci in D

b)If Ak is continuous-valued, then the attribute is typically assumed to have a Gaussian distribution

- A continuous-valued attribute is assumed to have a Gaussian distribution with a mean and standard deviation and is defined by

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}},$$

5. To classify an unknown sample X, P(X/Ci)P(Ci) is evaluated for each class Ci.

- The classifier predicts that the class label if and only if P(X/Ci)P(Ci) > P(X/C j)P(Cj) for 1≤ j ≤ m; j≠ I
- In other words, it is assigned to the class Ci for which P(X/Ci)P(Ci) is the maximum.

Table 6.1 Class-labeled training tuples from the *AllElectronics* customer database.

RID	*age*	*income*	*student*	*credit_rating*	*Class: buys_computer*
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

X=(age=youth,income=medium,student=yes,credit_rating=fair)

- P(buys_computer=yes)=9/14=0.643
- P(buys_computer=no)=5/14=0.357
- to compute conditional probalities P(X|Ci)
- P(age=youth|buys_computer=yes)=2/9=0.222
- P(age=youth|buys_computer=no)=3/5=0.600
- P(income=medium|buys_computer=yes)=4/9=0.444
- P(income=medium|buys_computer=no)=2/5=0.400
- P(student=yes|buys_computer=yes)=6/9=0.667

- P(student=yes|buys_computer=no)=1/5=0.200
- P(credit_ratiing=fair|buys_computer=yes)=6/9=0.667
- P(credt_rating=fair|buys_computer=no)=2/5=0.400

Using the probabilites

P(X|buys_computer=yes)=

P(age=youth|buys_computer=yes) *

P(income=medium|buys_computer=yes) *

P(student=yes|buys_computer=yes) *

P(credit_rating=fair|buys_computer=yes) *

=0.222*0.444*0.667*0.667= 0.044

P(X|buys_computer=no)=

P(age=youth|buys_computer=no) *

P(income=medium|buys_computer=no) *

P(student=yes|buys_computer=no) *

P(credit_rating=fair|buys_computer=no)

=0.600*0.400*0.200*0.400

=0.019

Similarly

P(X|buys_computer=yes)*P(buys_computer=yes)= 0.044*0.643=0.028

P(X|buys_computer=no)*P(buys_computer=no)

=0.019*0.357=0.007

X=(age=youth,income=medium,student=yes,credit_rating=fair)

Class is belonging to Yes

3.4 RULE-BASED CLASSIFICATION

Rule-Based Classification

In rule-based classifiers, the learned model is represented as a set of IF-THEN rules.

3.4.1 Using IF-THEN Rules for classification

- An IF-THEN rule is an expression of the form

IF *condition* THEN *conclusion.*

An example is rule *R*1,

*R*1: IF *age* = *youth* AND *student* = *yes* THEN *buys_computer* = *yes.*

- The “IF” part (or left side) of a rule is known as the **rule antecedent or precondition.**
- The “THEN” part (or right side) is the **rule consequent.**
- In the rule antecedent, the condition consists of one or more attribute tests (e.g., age = youth and student = yes) that are logically **ANDed.**

$$R1: (age = youth) \wedge (student = yes) \Rightarrow (buys_computer = yes).$$

- A rule R can be assessed by its coverage and accuracy.
- Given a tuple, X, from a class labeled data set, D, let ncovers be the number of tuples covered by R; ncorrect be the number of tuples correctly classified by R; and D be the number of tuples in D.
- We can define the coverage and accuracy of R as
- Consider rule R1, which covers 2 of the 14 tuples.
- It can correctly classify both tuples.
- Therefore, coverage(R1)= 2/14 = 14.28% and
- accuracy(R1)= 2/2 = 100%.
- Let’s see how we can use rule-based classification to predict the class label of a given tuple**, X.**
- If a rule is satisfied by **X,** the rule is said to be **triggered.**
- For example, suppose we have

$$X = (age = youth, income = medium, student = yes, credit_rating = fair)$$

- If R1 is the only rule satisfied, then the rule **fires** by returning the class prediction for **X.**
- If more than one rule is triggered, we need a **conflict** resolution strategy to figure out which rule gets to fire and assign its class prediction to **X**.
- There are two possible strategies., namely **size ordering** and **rule ordering.**
- The **size ordering** scheme assigns the highest priority to the triggering rule that has the “toughest” requirements,
- where toughness is measured by the rule antecedent size.
- That is, the triggering rule with the most attribute tests is fired.
- The **rule ordering** scheme prioritizes the rules beforehand.
- The ordering may be class-based or rule-based.
- With **class-based ordering,** the classes are sorted in order of decreasing “importance” such as by decreasing order of prevalence

3.4.2 Rule Extraction from Decision Tree

- Rule extraction is the process of converting a decision tree into a set of IF-THEN rules. Each rule corresponds to a path from the root node to a leaf node in the decision tree.

This is helpful because:

- It may be easier to read rules than follow a complex tree.

- Rules are human-readable and can often be used in expert systems.

How Rules Are Formed

- Every path from root → leaf becomes a rule.
- Conditions along the path are combined using AND.
- The class label at the leaf becomes the THEN part.

Extracted Rules:

R1: IF *age = youth*	AND *student = no*	THEN *buys_computer = no*
R2: IF *age = youth*	AND *student = yes*	THEN *buys_computer = yes*
R3: IF *age = middle_aged*		THEN *buys_computer = yes*
R4: IF *age = senior*	AND *credit_rating = excellent*	THEN *buys_computer = yes*
R5: IF *age = senior*	AND *credit_rating = fair*	THEN *buys_computer = no*

Key Characteristics of Extracted Rules

Property	Meaning
Mutually Exclusive	No two rules will fire for the same input (only one rule applies).
Exhaustive	Covers all possible input combinations (no need for default rule).
Unordered	Rules don't depend on order—any matching rule applies.

Rule Pruning (Simplification)

Once rules are extracted, we might want to prune (simplify) them:

- Remove unnecessary conditions.
- Keep only the parts that improve accuracy.
- Eliminate redundant or weak rules.

Example of Pruning:

From:

- IF age = senior AND student = yes AND credit_rating = excellent THEN class = yes

To:

- IF credit_rating = excellent THEN class = yes (if student/age don't improve accuracy)

After pruning:

- Rules may overlap (not mutually exclusive).
- Use class-based ordering (like in C4.5 algorithm).
- Prioritize rules that minimize false positives.

3.5 MODEL EVALUATION AND SELECTION

Model Evaluation

After building a classifier (e.g., for predicting customer behavior or medical diagnosis), we need to:

- Estimate its predictive accuracy
- Compare multiple classifiers
- Choose the best performing model

Measure	*Formula*
accuracy, recognition rate	$\frac{TP+TN}{P+N}$
error rate, misclassification rate	$\frac{FP+FN}{P+N}$
sensitivity, true positive rate, recall	$\frac{TP}{P}$
specificity, true negative rate	$\frac{TN}{N}$
precision	$\frac{TP}{TP+FP}$
F, F_1, F-score, harmonic mean of precision and recall	$\frac{2 \times precision \times recall}{precision + recall}$
F_β, where β is a non-negative real number	$\frac{(1+\beta^2) \times precision \times recall}{\beta^2 \times precision + recall}$

Figure: Evaluation measures.

3.5.1 Metrics for Evaluating Classifier Performance

CONFUSION MATRIX

- A confusion matrix summarizes the performance of a classification model by showing the correct and incorrect predictions made by the model compared to the actual values.

- This is the foundation of all performance metrics.

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	*TP*	*FN*	*P*
	no	*FP*	*TN*	*N*
	Total	P'	N'	$P + N$

True positives (TP): These refer to the positive tuples that were correctly labeled by the classifier. Let TP be the number of true positives.

True negatives(TN): These are the negative tuples that were correctly labeled by the classifier. Let TN be the number of true negatives.

False positives (FP): These are the negative tuples that were incorrectly labeled as positive (e.g., tuples of class buys computer = no for which the classifier predicted buys computer = yes). Let FP be the number of false positives.

False negatives (FN): These are the positive tuples that were mislabeled as negative (e.g., tuples of class buys computer = yes for which the classifier predicted buys computer = no). Let FN be the number of false negatives.

Classes	buys_computer = yes	buys_computer = no	Total	Recognition (%)
buys_computer = yes	**6954**	**46**	7000	99.34
buys_computer = no	**412**	**2588**	3000	86.27
Total	7366	2634	10,000	95.42

Figure :confusion matrix for the classes buys computer = yes and buys computer = no, where an entry in row i and column j shows the number of tuples of class i that were labeled by the classifier as class j. Ideally, the nondiagonal entries should be zero or close to zero.

1.Accuracy

- Measures how many instances were correctly classified overall.

$$accuracy = \frac{TP + TN}{P + N}.$$

Where:

- PPP = total actual positives (TP + FN)
- NNN = total actual negatives (TN + FP)

Also called: Overall recognition rate.

2. Error Rate (Misclassification Rate):

Shows how often the model is wrong.

$$error\ rate = \frac{FP + FN}{P + N}$$

3. Sensitivity (Recall or True Positive Rate)

- Measures the proportion of actual positives that were correctly identified.

$$sensitivity = \frac{TP}{P}$$

4. Specificity (True Negative Rate)

- Measures the proportion of actual negatives that were correctly identified.

$$specificity = \frac{TN}{N}$$

Example : Sensitivity and specificity. The Below Figure shows a confusion matrix for medical data where the class values are yes and no for a class label attribute, cancer. The sensitivity

Classes	*yes*	*no*	*Total*	*Recognition (%)*
yes	**90**	**210**	300	30.00
no	**140**	**9560**	9700	98.56
Total	230	9770	10,000	96.40

Figure : Confusion matrix for the classes cancer = yes and cancer = no

5. Accuracy as a Function of Sensitivity & Specificity

This formula shows how accuracy depends on sensitivity and specificity

$$accuracy = sensitivity\frac{P}{(P+N)} + specificity\frac{N}{(P+N)}$$

6.Precision

Precision measures how many of the predicted positive instances are actually positive.

$$precision = \frac{TP}{TP+FP}$$

7. Recall (also called Sensitivity or True Positive Rate)

- Recall measures how many of the actual positive cases were correctly predicted by the model.

$$recall = \frac{TP}{TP+FN} = \frac{TP}{P}$$

Where:

TP (True Positive): correctly predicted positives

FN (False Negative): actual positives wrongly predicted as negative

8. F1 Score

- The F1 score is the harmonic mean of Precision and Recall. It gives a single score that balances both concerns. It is especially useful when you need a balance between Precision and Recall.

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

9. Fβ Score

- The Fβ score is a generalization of the F1 score. It allows you to control how much weight you give to Recall versus Precision.

$$F_\beta = \frac{(1+\beta^2) \times precision \times recall}{\beta^2 \times precision + recall}$$

Key Metrics :

Metric	Formula	Meaning
Accuracy	accuracy = TP + TN /P + N	Overall correctness
Error Rate	error rate = FP + FN /P + N	Misclassification rate
Sensitivity / Recall	sensitivity = TP /P	Correct positive predictions
Specificity	specificity = TN/ N	Correct negative predictions
Precision	precision = TP /TP + FP	Exactness: how many predicted positives are correct
F1 Score	F = 2 × precision × recall /precision + recall	Harmonic mean of precision and recall
Fβ Score	Fβ = (1 + β 2) × precision × recall/ β 2 × precision + recall ,	Weighted F-measure

Additional Aspects to Compare Classifiers

1.**Speed**

- Refers to how fast a classifier can be trained and used for prediction.
- Important when real-time or quick decisions are needed.

2. **Robustness**

- Ability to handle noisy or incomplete data and still perform well.
- Evaluated by testing the model on increasingly noisy datasets.

3. Scalability

- How well a classifier can handle very large datasets.
- Evaluated using datasets of increasing size.

4. Interpretability

- How easily humans can understand the decision process of the model.
- Simple models like decision trees or rules are often more interpretable than complex models like neural networks.

Holdout Method for Estimating Accuracy

- The Holdout Method is a basic technique used to evaluate the performance of a classifier.

- It splits the data into:

Training Set: Used to train the model.

Test Set (Holdout Set): Used to evaluate the model's accuracy

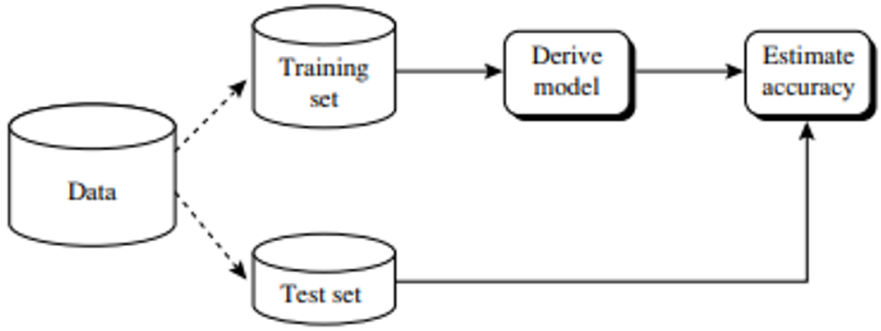


Figure : Estimating accuracy with the holdout method.

- Use **accuracy f**or balanced class distributions.
- Use r**ecall, precision, F1/Fβ** for imbalanced data.
- The **holdout method** is a fast way to test model generalization, but using cross-validation is more reliable for small datasets.

MCQs

Q1. The main goal of association analysis is:
a) Classification of data
b) Discovery of frequent patterns and relationships among items
c) Clustering of items
d) Regression prediction
Answer: b

Q2. Market Basket Analysis is a typical application of:
a) Classification
b) Regression
c) Association rule mining
d) Clustering
Answer: c

Q3. Apriori principle states that:
a) If an itemset is frequent, all of its supersets are frequent
b) If an itemset is infrequent, all of its supersets are infrequent
c) Both (a) and (b)
d) None of the above
Answer: b

Q4. Apriori principle helps in:
a) Rule evaluation
b) Candidate pruning
c) Classification
d) Clustering
Answer: b

Q5. In Apriori algorithm, candidate itemsets of size k are generated from:
a) Itemsets of size k+1
b) Itemsets of size k-1
c) Itemsets of size k
d) None of the above
Answer: b

Q6. Which measure is used to decide whether an itemset is frequent?
a) Support
b) Confidence
c) Lift
d) Correlation
Answer: a

Q7. Confidence-based pruning removes rules with:
a) Low support
b) Low confidence
c) High confidence
d) High support
Answer: b

Q8. Confidence measures:
a) How often items appear together in a dataset
b) Probability of Y occurring given X
c) Probability of X and Y occurring independently
d) None of these
Answer: b

Q9. In Apriori rule generation, rules are derived from:
a) Frequent itemsets
b) Infrequent itemsets
c) Random itemsets
d) None of the above
Answer: a

Q10. For the rule $X \rightarrow Y$, confidence is calculated as:
a) Support($X \cup Y$) / Support(Y)
b) Support($X \cup Y$) / Support(X)
c) Support(X) / Support($X \cup Y$)
d) Support(Y) / Support(X)
Answer: b

Q11. A maximal frequent itemset is:
a) A frequent itemset with no frequent supersets
b) An infrequent itemset with no frequent subsets
c) A subset of every frequent itemset
d) None of these
Answer: a

Q12. Maximal frequent itemsets help in:
a) Reducing the number of itemsets stored
b) Increasing support threshold
c) Increasing confidence threshold
d) None of these
Answer: a

Q13. A closed frequent itemset is:
a) A frequent itemset with no proper superset having the same support
b) An infrequent itemset with the same support
c) A superset of infrequent itemsets
d) None of these
Answer: a

Q14. Closed itemsets are useful because:
a) They give minimal redundancy
b) They maximize redundancy
c) They ignore support values
d) None of these
Answer: a

Q15. FP-Tree is used to:
a) Store all frequent itemsets compactly
b) Store infrequent itemsets
c) Replace clustering

d) Store classification trees
Answer: a

Q16. FP-Tree reduces:
a) Rule generation time
b) Candidate generation overhead
c) Support calculation
d) Both a and b
Answer: d

Q17. FP-Growth algorithm generates itemsets by:
a) Candidate generation and testing
b) Recursive pattern growth
c) Random selection
d) Hashing
Answer: b

Q18. Compared to Apriori, FP-Growth is:
a) Slower
b) More memory consuming
c) Faster and more efficient
d) None of these
Answer: c